

code.py -- Explainable Histopathology Image Classification

Source file: code.py — 491 lines

code.py -- Explainable Histopathology Image Classification | 1 / 8

```
1  """
2  Explainable Histopathology Image Classification
3  =====
4
5  A complete, single-file PyTorch pipeline for binary classification of
6  histopathology image patches (tumor vs. normal lymph-node tissue), with
7  Grad-CAM explainability.
8
9  WHAT THIS SCRIPT DOES
10 -----
11 1. Downloads raw whole-slide images (WSIs) + tumor annotation masks from the
12   public, unauthenticated AWS Open Data mirror of the CAMELYON16 challenge
13   (https://camelyon-dataset.s3.amazonaws.com/CAMELYON16/) -- the same lymph
14   node slide collection that the well-known PatchCamelyon (PCam) benchmark
15   was derived from.
16 2. Re-derives a small PCam-style dataset of labeled 96x96 tissue patches
17   directly from those raw slides: tissue detection, PCam-style
18   center-region labeling from the tumor mask, and Reinhard color
19   normalization to counter inter-slide staining variation.
20 3. Splits the data two ways:
21   - a pooled, patch-level stratified train/val/test split (6 slides)
22   - a slide-disjoint "cross-slide" holdout (2 further slides, never
23     touched during training) -- a stricter, more realistic test of
24     generalization to unseen tissue/staining.
25 4. Trains a compact CNN from scratch with augmentation.
26 5. Evaluates with ROC-AUC, precision, recall, F1, confusion matrix -- on
27   both the standard test split and the cross-slide holdout.
28 6. Produces Grad-CAM visual explanations for representative predictions.
29
30 This file is the consolidated, readable reference implementation. The
31 actual project is organized as separate modules under `src/`
32 (`extract_patches.py`, `model.py`, `gradcam.py`, `train.py`) which is how it
33 was developed and run; this file mirrors that same logic in one place for
34 easy reading end-to-end. See `report.md` for results and `learning_notes.md`
35 for a topic-by-topic explanation of every technique used here.
36 """
37
38 import os
39 import numpy as np
40 import cv2
41 import tifffile
42 import torch
43 import torch.nn as nn
44 import torch.nn.functional as F
45 from torch.utils.data import Dataset, DataLoader
46 import torchvision.transforms.v2 as T
47 from sklearn.metrics import (
48     roc_auc_score, roc_curve, precision_score, recall_score,
49     f1_score, confusion_matrix, classification_report,
50 )
51 import matplotlib.pyplot as plt
52
53 # -----
54 # Configuration
55 # -----
56 DATA_DIR = os.environ.get("WSI_DIR", "./data/wsi") # raw downloaded slides
57 OUT_DIR = os.environ.get("OUT_DIR", "./outputs") # patches + results
58 BASE_URL = "https://camelyon-dataset.s3.amazonaws.com/CAMELYON16"
59
60 PATCH = 96 # patch side length in pixels (PCam standard)
61 CENTER = 32 # PCam-style label decided by the central 32x32 region
62 TISSUE_SAT_THRESH = 0.08 # HSV saturation threshold separating tissue from glass
```

```

63 TISSUE_MIN_FRACTION = 0.35      # fraction of a patch that must look like tissue
64 MAX_POS_PER_SLIDE = 1800
65 MAX_NEG_PER_SLIDE = 1800
66
67 # Six slides feed a pooled, patch-level stratified train/val/test split.
68 # Two further slides are held out completely: they never contribute a
69 # single patch to train/val/test, so evaluating on them measures
70 # generalization to tissue/staining the model has genuinely never seen.
71 MAIN_SLIDES = [
72     ("tumor_091", True), ("normal_108", False),
73     ("tumor_075", True), ("normal_042", False),
74     ("tumor_082", True), ("normal_004", False),
75 ]
76 HOLDOUT_SLIDES = [("tumor_084", True), ("normal_150", False)]
77
78 # Reference LAB statistics for Reinhard color normalization, estimated once
79 # from ~400 tissue tiles of tumor_091. Every patch (every slide, every
80 # split) is remapped onto this single reference so that slide-to-slide
81 # staining/scanner color shift is not a shortcut feature the CNN can exploit.
82 REF_LAB_MEAN = np.array([136.72, 156.67, 102.10], dtype=np.float32)
83 REF_LAB_STD = np.array([61.34, 14.79, 14.73], dtype=np.float32)
84
85 IMAGENET_MEAN = [0.485, 0.456, 0.406]
86 IMAGENET_STD = [0.229, 0.224, 0.225]
87
88 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
89 torch.manual_seed(0)
90 np.random.seed(0)
91
92
93 # -----
94 # 1. Data acquisition -- download raw WSIs + masks from public S3
95 # -----
96 def download_slides():
97     """Download every slide/mask referenced by MAIN_SLIDES + HOLDOUT_SLIDES,
98     if not already present. No AWS credentials are needed; the CAMELYON16
99     mirror is a public, unauthenticated bucket."""
100     import urllib.request
101     os.makedirs(DATA_DIR, exist_ok=True)
102     files = []
103     for name, has_tumor in MAIN_SLIDES + HOLDOUT_SLIDES:
104         files.append(f"images/{name}.tif")
105         if has_tumor:
106             files.append(f"masks/{name}_mask.tif")
107     for rel in files:
108         local = os.path.join(DATA_DIR, os.path.basename(rel))
109         if os.path.exists(local):
110             continue
111         print(f"Downloading {rel} -> {local}")
112         urllib.request.urlretrieve(f"{BASE_URL}/{rel}", local)
113
114 # -----
115 # 2. Patch extraction from raw whole-slide TIFFs
116 # -----
117
118 def pick_level(tif_path, target_downsample=8.0):
119     """Pick the pyramid level whose downsample factor is closest to target.
120     WSIs are stored as multi-resolution pyramids (like a zoomable map); we
121     work at a downsampled level (~8x) instead of full resolution (which
122     would be tens of thousands of pixels per side)."""
123     tf = tiffio.TiffFile(tif_path)
124     base_shape = tf.pages[0].shape
125     best_i, best_diff = 0, 1e9
126     for i, page in enumerate(tf.pages):
127         diff = abs(base_shape[0] / page.shape[0] - target_downsample)
128         if diff < best_diff:
129             best_diff, best_i = diff, i
130     return best_i
131
132
133 def load_level(tif_path, level_idx):

```

```

134     return tiffiffle.TiffFile(tif_path).pages[level_idx].asarray()
135
136
137 def load_mask_matching(mask_path, target_shape):
138     """Tumor masks have their own pyramid, whose downsample steps don't
139     exactly line up with the image pyramid, so resize (nearest-neighbor,
140     since it's a label mask) onto the image's chosen level."""
141     tf = tiffiffle.TiffFile(mask_path)
142     best_i, best_diff = 0, 1e9
143     for i, page in enumerate(tf.pages):
144         diff = abs(page.shape[0] - target_shape[0]) + abs(page.shape[1] - target_shape[1])
145         if diff < best_diff:
146             best_diff, best_i = diff, i
147     mask = tf.pages[best_i].asarray()
148     if mask.shape[:2] != target_shape[:2]:
149         mask = cv2.resize(mask, (target_shape[1], target_shape[0]), interpolation=cv2.INTER_NEAREST)
150     return mask
151
152
153 def is_tissue(patch_rgb):
154     """Glass background is low-saturation (near-white); tissue is not."""
155     hsv = cv2.cvtColor(patch_rgb, cv2.COLOR_RGB2HSV)
156     sat = hsv[:, :, 1].astype(np.float32) / 255.0
157     return (sat > TISSUE_SAT_THRESH).mean() >= TISSUE_MIN_FRACTION
158
159
160 def reinhard_normalize(patch_rgb):
161     """Reinhard color transfer: standardize this patch's own LAB mean/std,
162     then rescale onto a single fixed reference distribution. Preserves the
163     patch's *internal* relative contrast (nuclei density/pattern -- the
164     diagnostic signal) while removing the *global* stain-intensity
165     difference between slides/scanners (the nuisance/batch-effect signal)."""
166     lab = cv2.cvtColor(patch_rgb, cv2.COLOR_RGB2LAB).astype(np.float32)
167     mean = lab.mean(axis=(0, 1))
168     std = lab.std(axis=(0, 1)) + 1e-6
169     lab = (lab - mean) / std * REF_LAB_STD + REF_LAB_MEAN
170     lab = np.clip(lab, 0, 255).astype(np.uint8)
171     return cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)
172
173
174 def extract_from_slide(name, has_tumor, rng):
175     """Tile one slide into non-overlapping 96x96 patches, keep only tissue
176     patches, and label each using the PCam convention: positive if the
177     *central* 32x32 region touches any tumor-annotated pixel; negative only
178     if the *entire* patch has zero tumor pixels (ambiguous boundary patches
179     are dropped rather than mislabeled)."""
180     img_path = os.path.join(DATA_DIR, f"{name}.tif")
181     level = pick_level(img_path)
182     img = load_level(img_path, level)
183     print(f"[{name}] level {level} shape {img.shape}")
184
185     mask = None
186     if has_tumor:
187         mask = load_mask_matching(os.path.join(DATA_DIR, f"{name}_mask.tif"), img.shape) > 0
188
189     h, w = img.shape[:2]
190     coords = [(r, c) for r in range(h // PATCH) for c in range(w // PATCH)]
191     rng.shuffle(coords)
192     c0 = (PATCH - CENTER) // 2
193
194     pos, neg = [], []
195     for r, c in coords:
196         y0, x0 = r * PATCH, c * PATCH
197         patch = img[y0:y0 + PATCH, x0:x0 + PATCH]
198         if patch.shape[:2] != (PATCH, PATCH) or not is_tissue(patch):
199             continue
200         if mask is not None:
201             m = mask[y0:y0 + PATCH, x0:x0 + PATCH]
202             if m[c0:c0 + CENTER, c0:c0 + CENTER].any():
203                 if len(pos) < MAX_POS_PER_SLIDE:
204                     pos.append(reinhard_normalize(patch))

```

```

205         continue
206     if m.any():
207         continue # tumor pixels exist but not in the center -> ambiguous, skip
208     if len(neg) < MAX_NEG_PER_SLIDE:
209         neg.append(reinhard_normalize(patch))
210     else:
211         if len(neg) < MAX_NEG_PER_SLIDE:
212             neg.append(reinhard_normalize(patch))
213     if len(pos) >= MAX_POS_PER_SLIDE and len(neg) >= MAX_NEG_PER_SLIDE:
214         break
215
216     print(f"[{name}] kept {len(pos)} positive / {len(neg)} negative patches")
217     return pos, neg
218
219
220 def build_pool(slide_list, rng):
221     all_x, all_y = [], []
222     for name, has_tumor in slide_list:
223         pos, neg = extract_from_slide(name, has_tumor, rng)
224         all_x.extend(pos + neg)
225         all_y.extend([1] * len(pos) + [0] * len(neg))
226     return np.stack(all_x).astype(np.uint8), np.array(all_y, dtype=np.int64)
227
228
229 def balance_classes(x, y, rng):
230     idx0, idx1 = np.where(y == 0)[0], np.where(y == 1)[0]
231     n = min(len(idx0), len(idx1))
232     keep = np.concatenate([rng.choice(idx0, n, replace=False), rng.choice(idx1, n, replace=False)])
233     rng.shuffle(keep)
234     return x[keep], y[keep]
235
236
237 def stratified_split(x, y, rng, fracs=(0.70, 0.15, 0.15)):
238     idx_splits = [[], [], []]
239     for c in np.unique(y):
240         idx_c = np.where(y == c)[0]
241         rng.shuffle(idx_c)
242         n_train = int(fracs[0] * len(idx_c))
243         n_val = int(fracs[1] * len(idx_c))
244         idx_splits[0].extend(idx_c[:n_train])
245         idx_splits[1].extend(idx_c[n_train:n_train + n_val])
246         idx_splits[2].extend(idx_c[n_train + n_val:])
247     out = []
248     for idx in idx_splits:
249         idx = np.array(idx)
250         rng.shuffle(idx)
251         out.append((x[idx], y[idx]))
252     return out
253
254
255 def build_dataset():
256     os.makedirs(OUT_DIR, exist_ok=True)
257     rng = np.random.default_rng(42)
258
259     x_pool, y_pool = build_pool(MAIN_SLIDES, rng)
260     x_pool, y_pool = balance_classes(x_pool, y_pool, rng)
261     (x_train, y_train), (x_val, y_val), (x_test, y_test) = stratified_split(x_pool, y_pool, rng)
262
263     x_holdout, y_holdout = build_pool(HOLDOUT_SLIDES, rng)
264     x_holdout, y_holdout = balance_classes(x_holdout, y_holdout, rng)
265
266     np.savez_compressed(
267         os.path.join(OUT_DIR, "patches.npz"),
268         x_train=x_train, y_train=y_train, x_val=x_val, y_val=y_val,
269         x_test=x_test, y_test=y_test, x_holdout=x_holdout, y_holdout=y_holdout,
270     )
271     print("train:", x_train.shape, "val:", x_val.shape,
272           "test:", x_test.shape, "holdout:", x_holdout.shape)
273
274
275 # -----

```

```

276 # 3. Model: a small CNN trained from scratch
277 # -----
278 class HistoCNN(nn.Module):
279     """VGG-style CNN: four conv blocks (32->64->128->256 channels) + global
280     average pool + a single linear head. The last block keeps its spatial
281     resolution (no pooling) so Grad-CAM has a reasonably sized feature map
282     (12x12) to compute a localization heatmap from."""
283
284     def __init__(self, num_classes=1):
285         super().__init__()
286         self.features = nn.Sequential(
287             self._block(3, 32),
288             self._block(32, 64),
289             self._block(64, 128),
290             self._block(128, 256, pool=False),
291         )
292         self.gap = nn.AdaptiveAvgPool2d(1)
293         self.classifier = nn.Sequential(nn.Flatten(), nn.Dropout(0.3), nn.Linear(256, num_classes))
294
295     @staticmethod
296     def _block(c_in, c_out, pool=True):
297         layers = [
298             nn.Conv2d(c_in, c_out, 3, padding=1), nn.BatchNorm2d(c_out), nn.ReLU(inplace=True),
299             nn.Conv2d(c_out, c_out, 3, padding=1), nn.BatchNorm2d(c_out), nn.ReLU(inplace=True),
300         ]
301         if pool:
302             layers.append(nn.MaxPool2d(2))
303         return nn.Sequential(*layers)
304
305     def forward(self, x):
306         return self.classifier(self.gap(self.features(x)))
307
308 # -----
309 # 4. Grad-CAM (Selvaraju et al., 2017)
310 # -----
311 class GradCAM:
312     """Weights each channel of a target conv layer's activation map by the
313     global-average-pooled gradient of the class score w.r.t. that channel,
314     then combines and ReLUs -- highlighting the regions that most increase
315     the model's predicted probability."""
316
317     def __init__(self, model, target_layer):
318         self.model = model
319         self.activations = self.gradients = None
320         target_layer.register_forward_hook(lambda m, i, o: setattr(self, "activations", o.detach()))
321         target_layer.register_full_backward_hook(lambda m, gi, go: setattr(self, "gradients", go[0].detach()))
322
323     def __call__(self, x, class_idx=None):
324         self.model.zero_grad()
325         logit = self.model(x)
326         score = logit[:, 0] if class_idx is None else logit[:, class_idx]
327         score.backward(retain_graph=True)
328         weights = self.gradients.mean(dim=(2, 3), keepdim=True)
329         cam = F.relu(weights * self.activations).sum(dim=1, keepdim=True)
330         cam = F.interpolate(cam, size=x.shape[2:], mode="bilinear", align_corners=False)[0, 0].cpu().numpy()
331         cam -= cam.min()
332         return cam / cam.max() if cam.max() > 0 else cam
333
334
335 def overlay_heatmap(rgb_uint8, cam, alpha=0.4):
336     heatmap = cv2.cvtColor(cv2.applyColorMap(np.uint8(255 * cam), cv2.COLORMAP_JET), cv2.COLOR_BGR2RGB)
337     return (alpha * heatmap + (1 - alpha) * rgb_uint8).astype(np.uint8)
338
339 # -----
340 # 5. Dataset / augmentation
341 # -----
342 class PatchDataset(Dataset):
343     def __init__(self, x, y, train=False):
344         self.x, self.y = x, y.astype(np.float32)

```

```

347     base = [T.ToImage(), T.ToDtype(torch.float32, scale=True), T.Normalize(IMAGENET_MEAN, IMAGENET_STD)]
348     if train:
349         # Geometric augmentation (H&E tissue has no canonical orientation)
350         # plus mild color jitter for residual, patch-level color noise
351         # left over after the slide-level Reinhard normalization above.
352         self.tf = T.Compose([
353             T.ToImage(),
354             T.RandomHorizontalFlip(0.5), T.RandomVerticalFlip(0.5), T.RandomRotation(90),
355             T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.05),
356             T.ToDtype(torch.float32, scale=True), T.Normalize(IMAGENET_MEAN, IMAGENET_STD),
357         ])
358     else:
359         self.tf = T.Compose(base)
360
361     def __len__(self):
362         return len(self.y)
363
364     def __getitem__(self, i):
365         return self.tf(self.x[i]), self.y[i]
366
367
368 # -----
369 # 6. Train / evaluate
370 # -----
371 def run_epoch(model, loader, criterion, optimizer=None):
372     train_mode = optimizer is not None
373     model.train(train_mode)
374     total_loss, probs_all, labels_all = 0.0, [], []
375     for xb, yb in loader:
376         xb, yb = xb.to(DEVICE), yb.to(DEVICE)
377         with torch.set_grad_enabled(train_mode):
378             logits = model(xb).squeeze(1)
379             loss = criterion(logits, yb)
380             if train_mode:
381                 optimizer.zero_grad(); loss.backward(); optimizer.step()
382             total_loss += loss.item() * len(yb)
383             probs_all.append(torch.sigmoid(logits).detach().cpu().numpy())
384             labels_all.append(yb.cpu().numpy())
385     probs, labels = np.concatenate(probs_all), np.concatenate(labels_all)
386     auc = roc_auc_score(labels, probs) if len(np.unique(labels)) > 1 else float("nan")
387     return total_loss / len(labels), auc
388
389
390 def predict(model, loader):
391     model.eval()
392     probs_all, labels_all = [], []
393     with torch.no_grad():
394         for xb, yb in loader:
395             probs_all.append(torch.sigmoid(model(xb.to(DEVICE))).squeeze(1).cpu().numpy())
396             labels_all.append(yb.numpy())
397     return np.concatenate(probs_all), np.concatenate(labels_all)
398
399
400 def evaluate_split(name, probs, labels, threshold, out_dir):
401     preds = (probs >= threshold).astype(int)
402     auc = roc_auc_score(labels, probs)
403     precision = precision_score(labels, preds, zero_division=0)
404     recall = recall_score(labels, preds, zero_division=0)
405     f1 = f1_score(labels, preds, zero_division=0)
406     cm = confusion_matrix(labels, preds)
407     report = classification_report(labels, preds, target_names=["normal", "tumor"], zero_division=0)
408     text = (f"=== {name} (threshold={threshold:.2f}) ===\nROC-AUC: {auc:.4f} Precision: {precision:.4f} "
409            f"Recall: {recall:.4f} F1: {f1:.4f}\nConfusion matrix [normal, tumor]:\n{cm}\n\n{report}\n")
410     print(text)
411
412     fpr, tpr, _ = roc_curve(labels, probs)
413     fig, ax = plt.subplots(1, 2, figsize=(10, 4))
414     ax[0].plot(fpr, tpr, label=f"AUC = {auc:.3f}"); ax[0].plot([0, 1], [0, 1], "--", color="gray")
415     ax[0].set_xlabel("FPR"); ax[0].set_ylabel("TPR"); ax[0].set_title(f"ROC ({name})"); ax[0].legend()
416     ax[1].imshow(cm, cmap="Blues")
417     ax[1].set_xticks([0, 1]); ax[1].set_xticklabels(["normal", "tumor"])

```

```

418 ax[1].set_yticks([0, 1]); ax[1].set_yticklabels(["normal", "tumor"])
419 ax[1].set_title(f"Confusion ({name})")
420 for i in range(2):
421     for j in range(2):
422         ax[1].text(j, i, str(cm[i, j]), ha="center", va="center",
423                    color="white" if cm[i, j] > cm.max() / 2 else "black")
424 fig.tight_layout(); fig.savefig(os.path.join(out_dir, f"roc_confusion_{name}.png"), dpi=140); plt.close(fig)
425 return text, preds
426
427
428 def train_and_evaluate():
429     data = np.load(os.path.join(OUT_DIR, "patches.npz"))
430     x_train, y_train = data["x_train"], data["y_train"]
431     x_val, y_val = data["x_val"], data["y_val"]
432     x_test, y_test = data["x_test"], data["y_test"]
433     x_holdout, y_holdout = data["x_holdout"], data["y_holdout"]
434
435     train_loader = DataLoader(PatchDataset(x_train, y_train, train=True), batch_size=32, shuffle=True, num_workers=2)
436     val_loader = DataLoader(PatchDataset(x_val, y_val), batch_size=64, num_workers=2)
437     test_loader = DataLoader(PatchDataset(x_test, y_test), batch_size=64, num_workers=2)
438     holdout_ds = PatchDataset(x_holdout, y_holdout)
439     holdout_loader = DataLoader(holdout_ds, batch_size=64, num_workers=2)
440
441     model = HistoCNN().to(DEVICE)
442     criterion = nn.BCEWithLogitsLoss()
443     optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
444     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="max", factor=0.5, patience=2)
445
446     best_val_auc, best_state = -1, None
447     for epoch in range(1, 21):
448         tr_loss, tr_auc = run_epoch(model, train_loader, criterion, optimizer)
449         val_loss, val_auc = run_epoch(model, val_loader, criterion)
450         scheduler.step(val_auc)
451         print(f"epoch {epoch:2d}/20 | train loss {tr_loss:.4f} auc {tr_auc:.4f} "
452               f"| val loss {val_loss:.4f} auc {val_auc:.4f}")
453         if val_auc > best_val_auc:
454             best_val_auc, best_state = val_auc, {k: v.cpu().clone() for k, v in model.state_dict().items()}
455     model.load_state_dict(best_state)
456     torch.save(model.state_dict(), os.path.join(OUT_DIR, "histo_cnn.pt"))
457
458     # Operating threshold is tuned on validation only, never on test/holdout.
459     val_probs, val_labels = predict(model, val_loader)
460     thresholds = np.linspace(0.05, 0.95, 91)
461     threshold = float(thresholds[np.argmax([f1_score(val_labels, (val_probs >= t).astype(int)) for t in thresholds])])
462
463     test_probs, test_labels = predict(model, test_loader)
464     holdout_probs, holdout_labels = predict(model, holdout_loader)
465     _, _ = evaluate_split("test", test_probs, test_labels, threshold, OUT_DIR)
466     _, holdout_preds = evaluate_split("holdout_cross_slide", holdout_probs, holdout_labels, threshold, OUT_DIR)
467
468     # Grad-CAM on one true-positive / true-negative / false-positive / false-negative
469     target_layer = model.features[3]
470     cam_engine = GradCAM(model, target_layer)
471     groups = {
472         "TP": np.where((holdout_labels == 1) & (holdout_preds == 1))[0],
473         "TN": np.where((holdout_labels == 0) & (holdout_preds == 0))[0],
474         "FP": np.where((holdout_labels == 0) & (holdout_preds == 1))[0],
475         "FN": np.where((holdout_labels == 1) & (holdout_preds == 0))[0],
476     }
477     picks = [(name, idx[0]) for name, idx in groups.items() if len(idx) > 0]
478     fig, axes = plt.subplots(2, len(picks), figsize=(3 * len(picks), 6))
479     for col, (name, idx) in enumerate(picks):
480         xb, _ = holdout_ds[idx]
481         cam = cam_engine(xb.unsqueeze(0).to(DEVICE))
482         overlay = overlay_heatmap(x_holdout[idx], cam)
483         axes[0, col].imshow(x_holdout[idx]); axes[0, col].set_title(f"{name} p={holdout_probs[idx]:.2f}"); axes[0, col].axis("off")
484         axes[1, col].imshow(overlay); axes[1, col].set_title("Grad-CAM"); axes[1, col].axis("off")
485     fig.tight_layout(); fig.savefig(os.path.join(OUT_DIR, "gradcam_examples.png"), dpi=140); plt.close(fig)
486
487
488 if __name__ == "__main__":

```

```
489     download_slides()
490     build_dataset()
491     train_and_evaluate()
```